

Domain 5: Context Management & Reliability

Domain 5 is 15% of the scored content. It is about keeping a Claude application accurate and dependable as a task grows long and complex. Long sessions, many tool calls, multiple agents, and multiple sources all put pressure on the model's limited working memory, and that pressure is where reliability problems appear. This domain teaches how to preserve the right information, escalate at the right moment, handle failures cleanly across agents, hold understanding together in large explorations, calibrate human review, and keep track of where each fact came from.

A. Foundations of Context Management and Reliability

Every Claude application works inside a context window, which is the model's working memory for a single request. It holds the system prompt, the conversation so far, every tool call and its output, and every file that has been read. The window is large but finite, so what goes into it has to be managed.

As a session runs, content piles up, and a fuller window does not stay as reliable as an empty one. Understanding why is the foundation for every technique in this domain, because most reliability problems in long-running agents trace back to how context was managed.

What is the Context Window?

It is everything the model can see at once when it generates its next response.

- It includes the system prompt, the conversation history, every tool call and result, and every file read.
- It has a fixed size, so adding more of one thing leaves less room for another.
- Current Claude models offer very large windows, with up to one million tokens available, but the window is still finite.

The context window is not a simple list where every item gets equal attention. As the window grows, the model must distribute its attention across more content. Items at the start and end of the window tend to receive stronger attention than items in the middle, a pattern known as the "lost in the middle" effect, which is covered in detail in section B.

Context Accumulation across a Session

Context grows with every turn, and tool-heavy work grows it fastest.

- Each user message, model reply, tool call, and tool result is added to the window.
- Tool results accumulate disproportionately: a few file reads or searches can fill a large share of the window with content you no longer need.
- Left unmanaged, old material crowds out the information the model needs right now.

Consider an agent investigating a customer billing issue. It might verify the customer identity (tool call + result), look up the order (tool call + result), check the refund policy (tool call + result), query the payment processor (tool call + result), and check the shipping status (tool call + result). After five tool

calls, the window may contain thousands of tokens of tool output, much of which is no longer needed for the current reasoning step.

Token Budgets

Treat the window as a budget you spend, not as free space.

- Every token of the system prompt, history, tool output, and file content spends part of the budget.
- When the budget runs low, you must clear, summarize, or offload content, or the session fails.
- System prompts and tool definitions are fixed costs that consume budget on every turn. The more tools you define, the less room remains for conversation and results.

Reliability and Context Growth

A fuller window is a less reliable window. This is the single most important idea in the section.

- **Context rot:** as the window fills, attention spreads thin, and the model is more likely to miss or confuse details.
- Important facts placed in the middle of a long context are the easiest to lose, a pattern covered in section B.
- Good context management is therefore a reliability technique, not just a cost technique.

The practical implication is that a session that worked perfectly with five tool calls may start making mistakes at fifteen, not because the model changed but because the context grew. Errors in long sessions should always be investigated as potential context management problems before concluding the prompt or the model is at fault.

When Context Rot Begins

Context rot does not start at a specific token count. It is a gradual degradation that becomes more noticeable as the window fills. The practical signs are:

- **At 20-30% utilization:** Performance is generally stable. The system prompt, tool definitions, and conversation history all fit comfortably.
- **At 50-70% utilization:** The model may start missing details from the middle of the context. Position-dependent recall issues appear.
- **At 80%+ utilization:** Performance degrades noticeably. The model may contradict earlier statements, miss instructions, or produce less coherent responses.
- **At 95%+ utilization:** The system is about to fail. Compaction or clearing is urgent.

These are approximate ranges, not hard thresholds. The key insight is that context management should happen proactively, before the window is full, not reactively after problems appear.

The Tools for Managing Context

Three current capabilities work together to keep the window healthy.

Tool	What it does	When to reach for it	Common Exam Trap
Context editing	Automatically clears stale tool results from the window	Tool-heavy work where old results are no longer	Assumed to also remember what it

Tool	What it does	When to reach for it	Common Exam Trap
		needed	cleared
Compaction	Summarizes the conversation and continues on the summary	Long single sessions approaching the limit	Expected to keep every detail; summaries lose some
Memory tool	Stores information in files outside the window	Facts that must survive clearing or span sessions	Skipped, so cleared information is lost for good

Context editing and compaction free up space, while the memory tool preserves what matters before space is freed, so the three are usually combined.

How they work together in practice: Before compacting a session, save durable facts (customer ID, open issue, constraints) to the memory tool. Then compact the conversation to free space. The compacted summary gives Claude the gist, and the memory tool supplies the exact values that the summary might have rounded off or dropped.

Multi-Turn Context Growth Rates

Different types of work consume context at different rates. Understanding the growth rate helps you plan when to clear or compact.

Work Type	Context Growth Rate	When to Manage
Text-only conversation	Low (hundreds of tokens per turn)	After many turns or when the session feels long
Tool-light tasks	Moderate (1-2K tokens per tool call)	After 10-15 tool calls
File-heavy exploration	High (1-5K tokens per file read)	After every few file reads
Search-heavy research	Very high (2-10K tokens per search)	After every search cluster

Signs That Context Needs Management

A few symptoms tell you the window is overloaded before it fails outright.

- The model repeats itself or forgets an instruction it was following earlier.
- Answers get slower, vaguer, or less accurate as the session goes on.
- Tool output dominates the window, and the actual task description is buried.
- You are near the token limit, which forces a clarification or a summary.
- The model starts contradicting earlier statements or "forgetting" decisions it already made.

Prompt Caching and Stable Context

Prompt caching lowers the cost and latency of reusing stable context, though it does not shrink the window itself.

- Cache stable prefixes, such as the system prompt and reference documents, so they are not reprocessed every turn.
- Put stable content first and changing content last, so the cached prefix stays valid across turns.
- Caching reduces cost and latency, but it is not a substitute for clearing or summarizing once the window fills.

The Stateless API and What It Means for Reliability

The Claude API is stateless. Each request is independent. The model does not remember the previous request unless you include the conversation history in the current request.

This has three important implications for reliability:

1. **You control what the model remembers.** If you omit conversation history, the model starts fresh. If you include it, the model has continuity. This is a feature, not a limitation, it means you can curate what the model sees.
2. **Clearing is your decision, not the model's.** The model cannot decide to forget old content. You must explicitly manage what stays in the conversation and what gets cleared, compacted, or moved to external storage.
3. **Re-grounding is your responsibility.** After compaction, the model's understanding of the conversation is based on the summary you provided. If the summary dropped a critical detail, the model does not know it was ever there. Re-injecting case facts after compaction is therefore a reliability requirement, not a convenience.

The Token Budget Analogy

One of the most useful mental models for the exam is to think of the context window as a financial budget. Every token of content you add is a dollar you spend.

- The system prompt is your fixed monthly rent, it is spent on every request.
- Tool definitions are your utility bills, they scale with the number of tools you connect.
- Conversation history is your discretionary spending, it grows with each turn.
- Tool results are your biggest variable expense, a single file read or search result can cost thousands of tokens.
- The memory tool is your savings account, information stored outside the window that you can access when needed.

When the budget runs low, you have three choices: clear stale spending (context editing), compress your spending history into a summary (compaction), or move funds to savings (memory tool). You cannot simply get a bigger budget and ignore the management problem, because context rot still applies, a fuller window is a less reliable window, regardless of how large it is.

Common Mistakes

- Treating the context window as free space instead of a finite budget.
- Letting tool results pile up until they crowd out the task.
- Assuming a bigger window removes the need to manage context when context rot still applies.

- Clearing content without first saving anything important to memory.
- Ignoring the accumulation of tool definitions across many connected MCP servers.

EXAM TIP: The exam may test scenarios where reliability drops in a long session, which points to context growth, not the prompt. Choose solutions that clear stale content and preserve critical facts in memory. Avoid assuming a larger context window removes the need to manage context.

Resources

- <https://www.anthropic.com/news/context-management>
 - <https://docs.claude.com/en/docs/build-with-claude/context-editing>
 - <https://docs.claude.com/en/docs/agents-and-tools/tool-use/memory-tool>
 - <https://docs.claude.com/en/docs/build-with-claude/prompt-caching>
-

B. Preserving Critical Information Across Long Interactions

In a long interaction, the goal is to keep the facts that matter available to the model while letting go of the chatter that does not. This section covers the failure modes that lose important information and the patterns that protect it.

Progressive Summarization and Its Risks

Summarizing is useful, but summarizing the same conversation again and again is lossy.

- Each summary pass can drop a detail that later turns out to matter, and once dropped, it is gone.
- A specific value, such as a date or an identifier, is easy to lose in a summary that keeps only the gist.
- Protect specific values by storing them as durable facts rather than trusting them to survive each summary.
- The risk compounds with each pass: the first summary loses 5% of details, the second loses 5% of what remains, and so on. After several passes, critical specifics have been replaced with generalities.

EXAM TIP: When a question describes a specific value (date, ID, constraint) that was present early in a conversation but is missing after several summarization passes, the answer points to a durable facts block, not to better summarization instructions.

The "Lost in the Middle" Effect

Models attend most reliably to the start and end of a long context and least reliably to the middle.

- **Position matters:** a key instruction buried in the middle of a long input is the most likely to be missed.
- Place the most important facts and instructions near the beginning or the end, not in the middle of a long block.
- This effect is well-documented in research on long-context language models and is confirmed by Anthropic's own long-context guidance.
- The effect gets stronger as the total context length increases. In a short prompt, position barely matters. In a 100,000-token prompt, position matters a great deal.

Durable Facts versus Passing History

Concept	Best Used For	Key Benefit	Common Exam Trap
Durable facts	Stable details needed for the whole task	Always available, never summarized away	Left in ordinary history and lost in a summary
Passing history	Turn-by-turn chatter and resolved steps	Can be trimmed or cleared safely	Kept in full, crowding out durable facts

The Case Facts Block: keep durable facts in one labeled block that you carry forward intact so they are never at the mercy of a summary. This is the most reliable way to anchor the details a task depends on.

Context Layers for Multiple Issues: when one conversation covers several separate issues, keep a separate context layer for each so facts from one issue do not bleed into another.

WORKED EXAMPLE

```
[CASE FACTS] (carry forward verbatim, do not summarize)
customer_id: 4471
plan: Business (annual)
open_issue: refund for duplicate charge on 2026-05-02
constraint: refunds over 100 require manager approval
[END CASE FACTS]
```

What this shows: the durable facts live in one clearly marked block that is carried forward unchanged. Even after the rest of the history is summarized or cleared, these values stay exact.

Tool Output Accumulation

Tool results are the fastest way to fill a window, so keep them lean.

- Trim verbose tool output before it accumulates, keeping only the fields you actually need.
- Use context editing to clear old tool results once they are no longer needed.
- When a tool returns a large JSON response with dozens of fields, extract the relevant fields and discard the rest before the result enters the conversation history.
- A single file read can consume thousands of tokens. If you only need one function from a 500-line file, extract that function rather than loading the entire file.

Position-Aware Input Ordering

Order input so the model reads the important parts where it attends best.

- Put key findings and instructions up front with clear section headers, or at the very end.
- Long reference material goes in the middle, where exact recall matters least.
- The case facts block belongs at the top of each new turn, where attention is strongest.

Structured Data versus Verbose Content between Agents

When one agent feeds another, send compact structured data, not raw reasoning.

- Have an upstream agent return structured data with key facts and citations instead of long prose, so a downstream agent with a small budget is not flooded.
- Structured handoffs preserve the facts that matter while spending far fewer tokens.
- A research agent that returns a 2,000-word prose analysis consumes far more downstream context than one that returns a 200-token structured summary with key findings, citations, and confidence scores.

Retention versus Retrieval

You do not have to keep everything in the window. Keep a little, and store the rest where it can be fetched on demand.

Concept	What it is	Key Benefit	Common Exam Trap
Retention	Keeping information in the window	Instant access, no lookup	Fills the window and triggers context rot
Retrieval	Storing information externally and fetching when needed	Keeps the active window small	Adds a lookup step and depends on good recall

Re-grounding after Compaction

A summary or a clear, exact detail can fade, so re-establish them before continuing.

- Re-inject the case facts block after compaction so durable values stay exact.
- Treat a summary as a starting point, not the full record, and restore anything the task depends on.

Common Mistakes

- Trusting specific values to survive repeated summarization instead of pinning them as durable facts.
- Burying a key instruction in the middle of a long input.
- Passing verbose agent-to-agent reasoning when structured data would do.
- Letting raw tool output accumulate unchecked.
- Forgetting to re-inject case facts after compaction.

Resources

- <https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/long-context-tips>
- <https://docs.claude.com/en/docs/build-with-claude/context-editing>
- <https://www.anthropic.com/news/context-management>

C. Escalation and Ambiguity Resolution Patterns

Escalation is handing a task to a human, and doing it at the right moment is a reliability skill. Escalate too late and you frustrate the user, escalate on the wrong signal and you waste human time. This section covers what should trigger an escalation and how to handle requests that are unclear.

What Triggers an Escalation?

Escalate on the substance of the situation, not on how the message feels.

Customer Requests for a Human: a direct request for a human is an immediate trigger. Honor it rather than trying to resolve the issue first.

Policy Gaps and Stalled Progress: escalate when the request falls outside policy, requires an authority the system does not have, or progress has genuinely stalled.

Authority Limits: escalate when the action requires an approval level the agent does not have, such as a refund above a threshold or a system change that requires human authorization.

Immediate Escalation versus Offered Resolution

Concept	Best Used For	Key Benefit	Common Exam Trap
Immediate escalation	A direct request for a human, or a hard policy limit	Respects the user and avoids friction	Delayed by trying to resolve first
Offered resolution	A frustrated user who has not asked for a human	May solve the issue faster than a handoff	Used to override an explicit request for a human

When a user explicitly asks for a human, escalate right away. When a user is upset but has not asked, you can acknowledge the frustration and offer to help, then escalate if they still want a human.

The Unreliability of Sentiment and Self-Rated Confidence

Two tempting signals are poor triggers, and the exam tests this directly.

- **Sentiment is unreliable:** a calm message can hide a hard problem, and a frustrated one can have an easy fix. Tone alone should not decide escalation.
- **Self-rated confidence is unreliable:** the model can be confidently wrong, so a high self-reported score is not proof the answer is right. Conversely, a low confidence score does not mean the answer is wrong, it may just indicate ambiguity in the source.
- Trigger on concrete conditions, such as an explicit request, a policy limit, or repeated failure, instead.

EXAM TIP: When a question offers sentiment analysis or self-rated confidence as the escalation trigger, these are distractors. The exam consistently tests that escalation should be driven by concrete conditions: explicit human requests, policy gaps, authority limits, or repeated failure.

Ambiguity and Clarification

When a request has more than one reasonable reading, ask rather than guess.

- If a query matches multiple records or interpretations, ask a short clarifying question instead of picking one.
- Guessing on ambiguity produces confident but possibly wrong action, which is worse than a brief question.

- The clarifying question should be specific: "I found two accounts matching that description, one under john@work.com and one under john@home.com. Which one?" is better than "Could you clarify?"

Multi-Step Escalation Paths

Not all escalations are binary. In production systems, there are often multiple levels of escalation, each appropriate for different situations.

Level	Trigger	Destination	Example
Level 0	Agent handles autonomously	No escalation	Standard refund within policy limits
Level 1	Agent needs human approval	Frontline support agent	Refund above auto-approval threshold
Level 2	Frontline cannot resolve	Specialist or team lead	Complex billing dispute spanning multiple months
Level 3	Specialist cannot resolve	Manager or policy exception	Request requires a policy exception not covered by any existing rule

The agent should route to the correct level directly when possible, rather than always starting at Level 1 and letting humans re-route.

WORKED EXAMPLE

```

Escalation Logic
if user explicitly asks for a human:
    escalate now # honor the request immediately
elif request is outside policy OR needs an authority we lack:
    escalate # policy gap
elif same step has failed repeatedly with no progress:
    escalate # stalled progress
elif request matches more than one record:
    ask one clarifying question # resolve ambiguity, do not guess
else:
    continue resolving

```

What this shows: escalation is driven by concrete conditions, an explicit request, a policy gap, or stalled progress, and ambiguity is handled by asking, not by sentiment or a confidence score.

Escalation with Context Handoff

When you do escalate, hand the human a warm start, not a blank slate. A well-structured escalation handoff includes five components:

1. **Reason for escalation:** Why the agent cannot continue (explicit human request, policy gap, authority limit, repeated failure).

2. **Case facts:** The durable facts from the case facts block (customer ID, account type, specific dates, amounts, constraints).
3. **Steps already taken:** A list of what the agent already tried and the results.
4. **Current state:** Where the issue stands right now — what is resolved, what is pending, what is blocked.
5. **Recommended next action:** The agent's assessment of what the human should do next.

Repeated Failure and Loop Detection

An agent that retries the same failing step forever is a reliability failure of its own.

- Track attempts per step, and after a set number of failures on the same step, stop and escalate.
- Detecting a loop early avoids burning tokens and time on an action that cannot succeed.

Common Mistakes

- Trying to resolve the issue after a user has explicitly asked for a human.
- Escalating on tone alone or refusing to escalate because the tone seems calm.
- Treating a high self-rated confidence as proof of correctness.
- Guessing an interpretation when a one-line question would remove the ambiguity.
- Escalating without context, forcing the human to start from scratch.

References

- <https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/be-clear-and-direct>
- <https://docs.claude.com/en/docs/agents-and-tools/tool-use/overview>

D. Error Propagation Across Multi-Agent Systems

In a multi-agent pipeline, errors in one agent affect every agent downstream. If a data-gathering agent fails silently, the synthesis agent produces a confident report with a hole in it, and nobody knows. This section is about making errors visible, categorized, and recoverable, so the system either fixes the problem locally or reports the gap honestly.

What a Useful Error Carries

A generic error like "something went wrong" tells the coordinator nothing. A useful error tells the coordinator what happened, why, and whether it is worth retrying.

Field	Purpose	Example
category	What type of failure	"access_failure", "timeout", "validation_error"
description	What specifically happened	"Database connection timed out after 30s"
retryable	Whether the same call might succeed on retry	true / false
source	Which agent or tool produced the error	"data_agent.query_billing"

EXAM TIP: When a question describes a coordinator that cannot determine what went wrong because the error message is too vague, the answer is structured errors with category, description, and retryability. Generic error messages like "tool failed" are always the wrong pattern.

Access Failure versus Valid Empty Result

This distinction is critical, and the exam tests it directly.

- An **access failure** means the tool could not reach its source, a timeout, a permission error, or a network failure. The absence of data is not meaningful because no query was successfully executed.
- A **valid empty result** means the tool successfully queried its source and found nothing. The absence of data is itself information, there genuinely is no matching record.
- If the system treats both the same way, a downstream agent might report "no data exists" when the truth is "we could not check."

Situation	What Happened	Correct Response	Wrong Response
API returned 500	Access failure	Return structured error with retryable: true	Return empty result
API returned 200 with zero records	Valid empty result	Return empty result with success status	Return error
API timed out	Access failure	Return structured error with retryable: true	Return empty result
API returned 200 with data	Successful result	Return the data	N/A

Error Categorization for Recovery Decisions

Different error categories require different recovery strategies. The coordinator's recovery logic should branch on the category, not on parsing the description text.

Error Category	Typical Cause	Recovery Strategy
timeout	Network latency, slow backend	Retry with backoff
rate_limited	Too many requests	Wait and retry after delay
permission_denied	Missing credentials or scope	Escalate — cannot retry
not_found	Resource does not exist	Valid result — not an error
validation_error	Malformed request	Fix the request and retry
service_unavailable	Backend is down	Try alternative source or annotate gap

Anti-Patterns in Error Handling

Two anti-patterns dominate the exam scenarios.

Silent suppression: An agent hits an error, catches it, and continues as if everything succeeded. The final report looks complete, but an entire data source is missing. This is the most dangerous anti-pattern because the output is confident and wrong, with no signal that anything failed.

Over-reaction: An agent hits one recoverable error and terminates the entire workflow. This wastes the progress of all other agents that succeeded. A timeout on one API call should not destroy a report that is 90% complete.

Local Recovery before Escalation

The correct pattern is to try to recover locally first and only escalate what cannot be fixed.

- If a tool call times out, retry it once or twice before reporting failure.
- If one source is unavailable, check whether an alternative source can provide the same information.
- If recovery fails, annotate the specific gap in the output rather than hiding it or killing the workflow.

Coverage Gap Annotation

When part of a report cannot be completed, say so explicitly in the output.

WORKED EXAMPLE

```
{
  "billing_analysis": {
    "status": "complete",
    "findings": [...]
  },
  "shipping_analysis": {
    "status": "incomplete",
    "reason": "Shipping API timed out after 3 retries",
    "recommendation": "Retry manually or check shipping status directly"
  }
}
```

What this shows: the report is honest about what it could and could not complete. A consumer of this report knows exactly which sections are reliable and which need follow-up.

Propagation Chains and Cascading Failures

In a multi-agent pipeline, errors do not stay local. A failure in one agent changes what the next agent receives, which changes what the agent after that receives, and so on. This is an error propagation chain.

The cascade pattern: Agent A fails → Agent B receives bad input → Agent B produces wrong output → Agent C synthesizes the wrong output into a confident final report. At no point in this chain did the system flag an issue, because each agent handled its input as if it were correct.

Breaking the chain: The structured error pattern breaks the chain by making failures visible at each handoff. When Agent A fails and returns a structured error instead of an empty result, Agent B can decide to skip that input, try an alternative, or annotate a coverage gap.

Partial Success and Coverage Honesty

Most real-world tasks can partially succeed. A report that covers 4 out of 5 data sources is more useful than no report at all, as long as the reader knows which source is missing.

The coverage honesty principle: When a task cannot be fully completed, report what was completed, what was not, and why. A partial result with honest annotations is more valuable than either a complete-looking result with hidden gaps or a total failure.

Common Mistakes

- Returning an empty result when the tool actually failed to reach its source.
- Suppressing errors silently and producing confident but incomplete output.
- Terminating the entire workflow over a single recoverable error.
- Using generic error messages that give the coordinator no basis for recovery decisions.
- Not including retryability information, so the coordinator does not know whether to retry or skip.

References

- <https://docs.claude.com/en/docs/agents-and-tools/tool-use/implement-tool-use>
 - <https://docs.claude.com/en/docs/agents-and-tools/tool-use/overview>
-

E. Context Management in Large Codebase Exploration

Exploring a large codebase with Claude Code is one of the most context-intensive tasks in production use. Each file read adds hundreds or thousands of tokens. A 15-file subsystem at 500 tokens per file consumes 7,500 tokens of raw code alone, and the model's reasoning about each file adds more. Without deliberate context management, the session degrades long before the exploration is complete.

Key Terms

- **Scratchpad file** is a persistent file on disk where Claude records key findings during long exploration sessions.
- **Subagent delegation** is spawning a focused subagent to read and analyze specific files, returning only a summary to the main session.
- **Exploration state** is the current understanding of the codebase: what has been read, what was found, what remains to investigate.
- **Progressive discovery** is the pattern of exploring a codebase in stages, where each stage's findings guide the next stage's focus.

The Problem: File Reads Fill the Window

In codebase exploration, the primary context consumers are file contents loaded by the Read tool.

- A single file read of a 200-line file might consume 800-1,200 tokens.
- An exploration that reads 20 files can consume 16,000-24,000 tokens of raw code.
- Add the model's reasoning about each file, and the total doubles.
- Once the window is full of old file contents, the model starts losing track of earlier findings.

Subagent Delegation for Verbose Reads

The most effective pattern for large codebase exploration is delegating verbose file reads to subagents.

- The main session holds the exploration strategy, the scratchpad, and the high-level understanding.
- Subagents are spawned to read specific files or file groups and return structured summaries.
- Only the summaries enter the main session's context, not the raw file contents.
- This keeps the main session's context clean and focused on reasoning, not on raw code.

Pattern	Main Session Context	Subagent Context	What Returns
Direct read	Full file contents (expensive)	N/A	N/A
Delegated read	Summary only (cheap)	Full file contents	Structured findings

When to Read Directly vs. Delegate

Not every file read needs a subagent. The decision depends on file size, how many files you need, and where you are in the session.

Situation	Best Approach	Reason
Small file (<100 lines), early in session	Read directly	Low cost, plenty of room
Large file (500+ lines)	Delegate to subagent	Returns summary, saves context
Multiple related files	Delegate one subagent for the group	Subagent can cross-reference within its own context
Targeted lookup (one function, one class)	Grep first, then read the specific section	No need to load the entire file
Late in a long session (window >60% full)	Always delegate	Context is too precious to spend on raw code

The Scratchpad Pattern

For long exploration sessions (30+ minutes), persist findings to an external file.

- Have Claude create and maintain a scratchpad file (e.g., `exploration-notes.md`) on disk.
- Record key findings, architectural decisions, class names, and important patterns as they are discovered.
- Re-read the scratchpad when starting a new phase of exploration to restore context.
- The scratchpad survives compaction, clearing, and even session boundaries.

The Exploration Journal Pattern

For complex, multi-session codebase explorations, the scratchpad evolves into an exploration journal — a structured document that tracks not just findings but also questions, hypotheses, and decisions.

Findings: Verified facts about the codebase (class names, design patterns, data flows). These are facts the exploration has confirmed.

Questions: Open questions that need investigation. Each question should note which files or areas might hold the answer.

Hypotheses: Educated guesses about how the system works that have not yet been verified. Marking something as a hypothesis prevents it from being treated as a confirmed finding.

Decisions: Architectural conclusions reached during exploration, with the reasoning that supports them. These are the outputs the exploration was started to produce.

WORKED EXAMPLE

Context Budget for a 15-File Subsystem

Direct reads (all 15 files): ~23,000 tokens of raw code + reasoning. Window fills quickly.

With subagent delegation: ~9,250 tokens of summaries + reasoning. Window stays clean.

Savings: Subagent delegation reduced main session context consumption by approximately 60%, leaving ample room for continued reasoning and additional investigation.

Progressive Discovery vs. Exhaustive Reading

Progressive discovery is the exploration pattern where each stage's findings guide the next stage's focus. It is the opposite of exhaustive reading, which tries to load and understand everything at once.

Progressive discovery works because:

- The first stage (entry points, base classes) tells you where to look next.
- Each subsequent stage is more targeted, reading only what the previous stage identified as relevant.
- Irrelevant files are never loaded, saving context for what matters.

Exhaustive reading fails because:

- Loading 15 files consumes most of the context window before any reasoning begins.
- The model has no room to think about what it read.
- Files loaded early fade from attention by the time later files are read.

Common Mistakes

- Reading every file into the main session, filling the window with raw code.
- Not persisting findings, so they are lost when the context is compacted.
- Exploring without a clear goal, reading files that turn out to be irrelevant.
- Not using Grep and Glob for targeted lookups when full file reads are unnecessary.

EXAM TIP: When a question describes an agent exploring a large codebase that starts losing earlier findings as the session progresses, the answer is persisting findings to a scratchpad or memory file and

delegating verbose reads to subagents. Not switching to a larger model, not clearing context periodically, and not pre-generating file summaries.

References

- <https://code.claude.com/docs/en/sub-agents>
 - <https://code.claude.com/docs/en/how-claude-code-works>
 - <https://docs.anthropic.com/en/docs/claude-code/best-practices>
-

F. Human Review Workflows and Confidence Calibration

Key Terms

- **Confidence threshold** is the score below which output is routed to human review instead of being accepted automatically.
- **Stratified sampling** is measuring accuracy within each segment (document type, category, source) rather than across the whole dataset.
- **Aggregate accuracy** is the single overall accuracy number, which can hide a segment that is failing badly.
- **Human-in-the-loop** is a workflow where a human reviews and approves output before it enters the downstream system.

When to Route to Human Review

Route to human review when the system's output is uncertain, novel, or high-stakes.

- **Low confidence:** When the model signals uncertainty about an extraction or a decision, route it for human verification.
- **Novel input:** When the input type has not been seen before (a new document format, a new customer segment), the system has no basis for calibration.
- **High stakes:** When the cost of an error is high (financial transactions, medical records, legal documents), human review is a safety net regardless of confidence.
- **Conflicting sources:** When the system detects contradictory information from multiple sources, a human can resolve what the model cannot.

Setting Confidence Thresholds

A confidence threshold defines the boundary between automatic acceptance and human review.

- Set the threshold based on the cost of errors in your domain. Financial applications need higher thresholds than internal summaries.
- The threshold is not permanent. Start conservative (routing more to humans), measure the accuracy of what passes automatically, and adjust.
- Track the volume of records routed to review. If the review queue is overwhelmed, the threshold may be too strict, or the model may genuinely be struggling with that input type.

WORKED EXAMPLE

Confidence Threshold Calibration: An invoice extraction system starts with a confidence threshold of 0.85. After one month of production data: Records above 0.85: 92% accuracy (acceptable). Records below 0.85: 64% accuracy (confirming the threshold catches weak extractions). Review queue volume: 15% of total records (manageable). The threshold is working: it routes uncertain records to humans while keeping the queue manageable. If accuracy above the threshold dropped, the threshold would need to rise.

The Hidden Weakness Problem

A single accuracy number can hide a failing segment.

- An extraction system reports 96% overall accuracy. The team is satisfied. But accuracy by document type tells a different story:
 - Standard invoices: 99%
 - Scanned paper invoices: 79%
 - Handwritten receipts: 68%
- The 96% aggregate is dominated by the high-volume standard invoices. The rare types are failing badly, but the overall number masks it.

Stratified Sampling

The solution is to measure accuracy within each segment, not just across the whole dataset.

- Segment by document type, source, category, or any dimension where performance might vary.
- Measure each segment separately. A 72% on handwritten receipts is actionable, you can add examples, adjust the schema, or route all handwritten receipts to review.
- Random sampling alone does not solve this because rare types are underrepresented. Stratified sampling ensures each type is measured on its own.

Sampling Method	What It Measures	What It Misses
Random sampling	Overall accuracy across the dataset	Weak segments hidden by strong segments
Stratified sampling	Accuracy within each segment	Nothing, if segments are defined well

WORKED EXAMPLE

Stratified Accuracy Reveals Hidden Weakness: Standard PDF invoices (80% of volume): 98% accuracy. Scanned paper invoices (12% of volume): 79% accuracy. Handwritten receipts (8% of volume): 68% accuracy. Aggregate: 94% — looks acceptable.

What the aggregate hides: customers who submit handwritten receipts experience extraction errors on one-third of their documents. Stratified sampling surfaces the exact problem areas.

Review Queue Management

Routing low-confidence output to human review creates a queue that must be managed to stay useful.

- **Queue volume monitoring:** If more than 25% of output goes to review, the threshold is too strict or the model is struggling with the input type.
- **Review turnaround time:** If the review queue grows faster than reviewers can process it, either adjust the threshold or add reviewers.
- **Feedback loop:** Use review outcomes to improve the system. If reviewers consistently approve a specific pattern that scores low, the pattern might be safe to auto-accept.
- **Reviewer agreement:** Measure whether reviewers agree with each other. Low inter-rater agreement means the task is inherently ambiguous, and the review process itself may need better guidelines.

Common Mistakes

- Trusting a single aggregate accuracy number without looking at segments.
- Setting a confidence threshold once and never adjusting it.
- Routing everything to human review, which defeats the purpose of automation.
- Using random sampling for a dataset with rare but important edge cases.

EXAM TIP: When a question describes high overall accuracy but a specific document type or category that is failing, the answer is stratified sampling by segment, not increasing the overall sample size or trusting the aggregate number.

References

- <https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/reduce-hallucinations>
- <https://docs.claude.com/en/docs/build-with-claude/structured-outputs>

G. Information Provenance in Multi-Source Synthesis

When a system gathers information from multiple sources and synthesizes it into one output, the question becomes, "Where did each fact come from?" Provenance is the answer. Without it, the synthesis is unverifiable, conflicts are invisible, and errors are untraceable.

Key Terms

- **Provenance** is the record of where each piece of information came from.
- **Claim-source mapping** is the link between a specific claim in the output and the source(s) that support it.
- **Source conflict** is when two or more sources provide different values for the same fact.
- **Publication date** is the date a source was published, which can resolve conflicts when one source simply updates another.
- **Attribution** is marking each claim with its source so a reader can verify it.
- **Invented consensus** is when the model silently resolves a conflict by picking one value or averaging, presenting the result as if all sources agree.

Why Provenance Matters

Without provenance, a synthesis cannot be verified, updated, or trusted.

- A report that says "revenue grew 12% last quarter" is useful. A report that says "revenue grew 12% last quarter (Q3 2026 earnings report, p. 4)" is verifiable.
- When a source turns out to be wrong, provenance tells you which claims in the synthesis are affected.
- When a downstream consumer needs to update a figure, provenance tells them where the original came from.

Claim-Source Mappings

Every factual claim in a synthesis should link back to the source that supports it.

- Use structured output with a claims array where each entry carries the claim text, the source identifier, and optionally a page number or section reference.
- If a claim is supported by multiple sources, list all of them.
- If a claim is not supported by any source, it should not appear in the synthesis — or it should be flagged as the model's own inference.

WORKED EXAMPLE

```
{
  "claims": [
    {
      "text": "The global AI market is projected to reach $500B by 2028",
      "sources": [
        {
          "id": "gartner_2026",
          "title": "AI Market Forecast 2026",
          "date": "2026-03-15",
          "page": 12
        }
      ]
    },
    {
      "text": "Enterprise AI adoption reached 72% in 2025",
      "sources": [
        { "id": "mckinsey_survey", "date": "2025-11-20" },
        { "id": "idc_report", "date": "2026-01-10" }
      ],
      "note": "IDC reports 74% – slight difference likely due to sample timing"
    }
  ]
}
```

What this shows: each claim carries its sources, and a conflict between two sources is annotated rather than silently resolved.

Handling Source Conflicts

When two sources disagree, the system should not silently pick one.

Using Publication Dates: A 2026 source that updates a 2024 figure is not a conflict but rather a correction. Check dates first.

Annotating Genuine Conflicts: When two contemporaneous sources disagree, present both values with their sources and let the consumer decide.

Never Average: Averaging two conflicting figures produces a number that no source reported. It looks precise and is entirely fabricated.

Conflict Pattern	Resolution	Anti-Pattern
Newer source updates older	Use the newer value, note the update	Use the older value
Two contemporaneous sources disagree	Annotate both values with sources	Pick one silently
One source has higher authority	Note the authority difference	Average the values
Sources use different definitions	Note the definitional difference	Treat them as the same metric

The Source Authority Hierarchy

When evaluating conflicting sources, not all sources carry equal weight.

Primary sources are the original producers of information: official reports, direct measurements, and first-party data. These carry the most weight.

Secondary sources interpret or aggregate primary sources: news articles, analyst reports, and review papers. They are useful but can introduce errors through interpretation.

Tertiary sources compile information from secondary sources: encyclopedias, directories, and aggregator sites. They are convenient but furthest from the original data.

Source Type	Authority	Examples	Use When
Primary	Highest	SEC filings, official reports, first-party data	Available and current
Secondary	Medium	News articles, analyst reports	Primary not available or for context
Tertiary	Lowest	Aggregator sites, encyclopedias	Quick reference only

Multi-Agent Provenance Preservation

In a coordinator-subagent architecture, provenance must survive the handoff between agents.

The source laundering problem: Agent A extracts a claim from Source X with full attribution. Agent A passes the claim to the coordinator. The Coordinator passes it to Agent B for synthesis. By the time Agent

B includes it in the final report, the attribution to Source X has been dropped. The report presents the claim as a general fact with no source.

The fix: Every inter-agent handoff must preserve the claim-source mapping. Subagent outputs should use a structured format with source attribution fields. The coordinator should merge subagent outputs while preserving attributions, not summarizing them into prose. The final synthesis should trace each claim back to the original source, not just to "the search agent."

Anti-Patterns in Provenance

Dropping attribution entirely: A synthesis that presents facts without sources is unverifiable.

Invented consensus: When the model encounters a conflict, it silently picks the "most reasonable" value and presents it as the agreed-upon figure. The output reads as if all sources agree, when they do not.

Source laundering: When one agent passes a fact to another without attribution, and the downstream agent presents it as its own finding.

Common Mistakes

- Dropping source attribution in agent-to-agent handoffs.
- Silently resolving conflicts by picking one value or averaging.
- Presenting model inferences as if they were sourced claims.
- Not checking publication dates before treating a disagreement as a conflict.

EXAM TIP: When a question describes a report where two sources give different figures for the same metric and the report shows only one number with no attribution — the answer is to preserve claim-source mappings, annotate the conflict, and use dates to interpret it. Never average, never silently pick one, never drop the metric entirely.

References

- <https://docs.claude.com/en/docs/build-with-claude/structured-outputs>
- <https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/reduce-hallucinations>

Worked Examples Across Domain 5

Worked Example: Error Propagation in a Research Pipeline

A multi-agent research system has a search agent, an analysis agent, and a synthesis agent. The search agent queries three sources. Source A returns data, Source B returns a valid empty result, and Source C times out.

Wrong pattern: Search agent returns combined results from A and B only, with no mention of C. Synthesis agent produces a report that appears to cover all sources. The report is missing Source C's data with no indication.

Correct pattern: Search agent returns results from A, notes that B had no matching data (valid empty), and reports that C failed with a structured error (timeout, retryable). Synthesis agent includes A's data,

notes B had no data, and annotates that Source C could not be reached. The final report is honest about coverage.

Worked Example: Escalation with Warm Handoff

A customer contacts support about a billing dispute. The agent verifies the customer, looks up the order, and checks the refund policy. The refund amount exceeds the auto-approval limit, requiring human authorization.

Wrong pattern: The agent says "I'm transferring you to a human agent" with no context. The human agent asks the customer to repeat everything.

Correct pattern: The agent escalates with a structured handoff:

```
{
  "reason": "refund_exceeds_auto_approval_limit",
  "case_facts": {
    "customer_id": "C-4471",
    "order_id": "0-88421",
    "charge_date": "2026-05-02",
    "refund_amount": 247.50,
    "auto_approval_limit": 100.00
  },
  "steps_taken": [
    "Customer identity verified via email match",
    "Order 0-88421 confirmed as duplicate charge",
    "Refund policy checked: amount exceeds $100 auto-approval"
  ],
  "recommended_action": "Approve refund of $247.50 for duplicate charge"
}
```

The human agent receives the durable facts, the investigation history, and a recommendation — the conversation continues instead of restarting.

Worked Example: Stratified Accuracy Audit

An invoice extraction system processes three document types. The team measures overall accuracy at 94% and considers the system production-ready.

Document Type	Volume	Accuracy	Contribution to Aggregate
Standard PDF invoices	80%	98%	Dominates the aggregate
Scanned paper invoices	12%	79%	Hidden by low volume
Handwritten receipts	8%	68%	Completely hidden
Aggregate	100%	94%	Looks acceptable

What the aggregate hides: Customers who submit handwritten receipts experience extraction errors on one-third of their documents. The 94% looks healthy, but two segments are failing badly.

What stratified sampling reveals: Each type measured separately surfaces the exact problem areas. The team can now add few-shot examples for scanned invoices, create a dedicated extraction prompt for handwritten receipts, or route both types to human review until the model improves.

Worked Example: Context Budget Planning

A developer needs to understand a caching subsystem with 15 files totaling approximately 8,000 lines of code.

Direct reads (all 15 files loaded into main session):

Component	Estimated Tokens	Notes
System prompt + tool definitions	~3,000	Fixed cost, every turn
15 files at ~800 tokens each	~12,000	All loaded directly
Model reasoning per file	~6,000	~400 tokens of reasoning per file
Conversation history	~2,000	Growing with each turn
Total	~23,000	Exceeds comfortable range

With subagent delegation:

Component	Estimated Tokens	Notes
System prompt + tool definitions	~3,000	Fixed cost
15 subagent summaries at ~150 tokens each	~2,250	Only summaries enter main context
Model reasoning on summaries	~2,000	Reasoning on compact data
Conversation history + scratchpad reads	~2,000	Includes re-reading scratchpad
Total	~9,250	Well within comfortable range

Savings: Subagent delegation reduced main session context consumption by approximately 60%.

Worked Example: Temporal Conflict Resolution

A research agent finds two sources reporting different values for the same metric:

- Source A (published March 2024): "Global cloud spending reached \$480 billion in 2023."
- Source B (published January 2026): "Global cloud spending reached \$520 billion in 2023, revised upward from earlier estimates."

Wrong pattern: Average the two figures to get \$500 billion. This number appears in neither source and is fabricated.

Wrong pattern: Pick Source A because it was published closer to 2023. Source B explicitly states it is a revision.

Correct pattern: Use Source B's revised figure (\$520 billion) and annotate: "Revised upward from \$480B (Source A, March 2024) to \$520B (Source B, January 2026). Source B explicitly notes the revision." This preserves provenance, explains the discrepancy, and gives the reader full context.

Worked Example: Exploration Journal

A developer is exploring a caching subsystem. After two hours of investigation, the exploration journal on disk contains:

Findings:

- CacheManager is the base class for all cache implementations (src/cache/manager.py)
- Redis is the primary cache backend; PostgreSQL is used as fallback
- Invalidation uses a write-through pattern with TTL-based expiry
- Cache keys follow the pattern: {entity_type}:{entity_id}:{field}

Questions:

- How does cache stampede prevention work? (check src/cache/locks.py)
- What happens when Redis is unavailable? (check fallback logic in manager.py)
- Are cache keys namespaced per tenant? (check multi-tenant config)

What this shows: The journal separates confirmed findings from hypotheses, tracks open questions with pointers to where answers might be, and records architectural decisions with their evidence. This survives compaction and session boundaries.

Domain 5 Services Appendix

Context Management Reference

Tool	What It Does	When to Use
Context editing	Clears stale tool results from the window	Tool-heavy sessions with accumulating results
Compaction	Summarizes conversation and continues on summary	Long sessions approaching the token limit
Memory tool	Stores facts in files outside the window	Facts that must survive clearing or span sessions
Prompt caching	Caches stable prefixes for cost/latency savings	Repeated requests with the same system prompt

Escalation Trigger Reference

Trigger	Action	Reliability
Explicit human request	Escalate immediately	High: Always honor
Policy gap or authority limit	Escalate	High: Concrete condition

Trigger	Action	Reliability
Repeated failure on same step	Escalate	High: Loop detection
Ambiguous request	Ask clarifying question	High: Resolves ambiguity
Sentiment (frustration)	Do not escalate on this alone	Low: Unreliable signal
Self-rated confidence	Do not use as trigger	Low: Poorly calibrated

Error Response Structure

Field	Purpose	Example Values
category	Type of failure	"access_failure", "timeout", "validation_error", "permission_denied"
description	What happened	"Database connection timed out after 30s"
retryable	Whether retry might succeed	true / false
source	Which component failed	"data_agent.query_billing"

Provenance Output Structure

Field	Purpose
claims[].text	The factual claim
claims[].sources[]	Source(s) supporting the claim
claims[].sources[].id	Unique source identifier
claims[].sources[].date	Publication date
claims[].note	Conflict annotation, if applicable

Codebase Exploration Reference

Pattern	Main Context Cost	When to Use
Direct file read	High (full file contents)	Small files, quick lookups
Subagent delegation	Low (summary only)	Large files, multi-file exploration
Grep search	Minimal (matching lines only)	Finding specific patterns
Glob search	Minimal (file paths only)	Mapping directory structure
Scratchpad file	None (on disk)	Persisting findings across compaction

Source Authority Hierarchy

Source Type	Authority	Examples
Primary	Highest	SEC filings, official reports, first-party data
Secondary	Medium	News articles, analyst reports
Tertiary	Lowest	Aggregator sites, encyclopedias

Domain 5: Context Management & Reliability — Sample Questions

Question 1

An AI Engineer is responsible for managing a production Claude system that needs to maintain high availability. The system comprises multiple microservices that interact with one another, and it's crucial to track every request, process, and error for debugging, monitoring, and auditing. As part of your workflow, the engineer must ensure that logging is configured properly to capture all necessary events without overwhelming the system with excessive log data.

Which of the following is the most effective approach to implement comprehensive logging in your production Claude system?

1. Log every request, response, and error in great detail, including sensitive data such as user passwords, for full transparency.
2. Implement logging at key points within each microservice, ensure that only relevant information is logged, and sensitive data is omitted.
3. Rely on basic error logs and only log the highest-level system failures to reduce log volume.
4. Implement logging at the microservice level, but store logs locally on each instance to avoid central aggregation for performance reasons.

Correct Answer: 2

Explanation:

Production AI systems that use multiple microservices require structured and efficient logging to support monitoring, debugging, auditing, and incident response. Since requests and processes often span multiple services, logging must provide sufficient visibility to trace system activity while preserving performance and security. Effective logging strategies, therefore, focus on capturing meaningful operational events without exposing sensitive information or generating unnecessary log volume.

Implementing logging at key points in each microservice, while ensuring only relevant information is recorded, is considered the most effective approach. This method allows engineers to track requests, monitor workflows, and diagnose failures efficiently while maintaining system performance. Omitting sensitive data, such as passwords, authentication tokens, or personal information, is also essential, as secure logging practices help reduce privacy and security risks. By focusing only on meaningful operational details, organizations can maintain useful observability without overwhelming storage and monitoring systems.

Modern distributed systems typically rely on centralized and structured logging practices to improve reliability and operational awareness. Logging relevant events across microservices supports troubleshooting, auditing, and performance analysis while still maintaining compliance with security and

privacy standards. A balanced logging strategy helps organizations achieve visibility into system behavior while avoiding excessive or unsafe data collection.

Hence, the correct answer is: **Implement logging at key points within each microservice, ensure that only relevant information is logged, and sensitive data is omitted.**

The option that says: *Log every request, response, and error in great detail, including sensitive data such as user passwords, for full transparency* is incorrect because logging highly sensitive information such as passwords can create serious security and compliance risks. Comprehensive logging should primarily focus on operational visibility while protecting confidential data.

The option that says: *Rely on basic error logs and only log the highest-level system failures to reduce log volume* is incorrect because relying only on basic high-level error logs may simply provide insufficient detail for debugging distributed microservice interactions. Effective monitoring typically requires visibility into important service-level events and workflows.

The option that says: *Implement logging at the microservice level, but store logs locally on each instance to avoid central aggregation for performance reasons* is incorrect because storing logs only locally can reduce centralized visibility and make troubleshooting across multiple services more difficult. Modern production systems usually benefit from centralized or aggregated logging for easier analysis and monitoring.

References:

- <https://platform.claude.com/docs/en/build-with-claude/overview>
- <https://mcpmarket.com/tools/skills/audit-logging-protocol>
- <https://code.claude.com/docs/en/security>

Question 2

A financial services company is deploying a Claude-based platform for processing sensitive customer requests and internal workflows. To meet strict auditing and regulatory requirements, the organization needs to trace all user interactions, workflow executions, tool invocations, and administrative actions. A recent compliance review revealed missing critical records, making it difficult to reconstruct several workflow activities. The architecture team must implement a solution to enhance traceability, accountability, and audit readiness.

Which is the most effective approach for compliance logging in this Claude-based system?

1. Log all actions with timestamps to create a complete audit trail of system activity and workflow execution.
2. Store only error-related events to reduce storage costs and minimize operational overhead.
3. Retain logs temporarily in local application memory and export only during incidents.
4. Log workflow summaries at the end of each session instead of recording individual operations and events.

Correct Answer: 1

Explanation:

Compliance-focused systems require detailed audit trails that accurately record system activity over time. Logging all actions with timestamps provides a chronological record of user operations, tool executions, workflow events, configuration changes, and administrative activities. These records enable organizations to reconstruct events during investigations, validate compliance controls, and demonstrate accountability during audits.

Timestamped logs are especially important in distributed Claude architectures where workflows may span multiple services, agents, and external tools. Precise timestamps help correlate events across systems, identify the sequence of operations, and support forensic analysis when investigating failures or suspicious behavior. Comprehensive logging also improves operational visibility and incident response capabilities.

In regulated environments, incomplete logging creates gaps in traceability that can lead to failed audits, operational risk, and compliance violations. Centralized, timestamped audit records help organizations maintain security governance, support regulatory reporting, and preserve historical evidence of system activity. Effective compliance logging, therefore, requires both completeness and accurate event timing.

Hence, the correct answer is: **Log all actions with timestamps to create a complete audit trail of system activity and workflow execution.**

The option that says: *Store only error-related events to reduce storage costs and minimize operational overhead* is incorrect because compliance audits typically require complete operational traceability rather than only failure-related events.

The option that says: *Retain logs temporarily in local application memory and export only during incidents* is incorrect because temporary in-memory storage simply increases the risk of losing important audit records during crashes or system failures.

The option that says: *Log workflow summaries at the end of each session instead of recording individual operations and events* is incorrect because summary-level logging primarily lacks the detailed event granularity needed for compliance investigations and forensic analysis.

References:

- <https://code.claude.com/docs/en/best-practices>
- <https://platform.claude.com/docs/en/agents-and-tools/tool-use/overview>

Additional Exam Guidance for Domain 5

How Domain 5 Connects to Other Domains

Domain 5 concepts appear throughout the exam, not just in questions labeled "Context Management." Understanding these connections helps you recognize Domain 5 patterns in scenarios that seem to belong to other domains.

Connection to Domain 1 (Agentic Architecture): Long-running agentic loops accumulate context with every tool call. The `max_turns` and `max_budget_usd` flags from Domain 3 are cost controls, but context management is the reliability control. An agent that runs for 20 turns without clearing stale tool results will start making mistakes, that is a Domain 5 problem even though the agent was designed in Domain 1.

Connection to Domain 2 (Tool Design): Tool results that return verbose, unfiltered data are a Domain 5 problem. A well-designed tool that returns only the fields the agent needs is both a Domain 2 design choice and a Domain 5 reliability measure.

Connection to Domain 3 (Claude Code): Scratchpad files and sub-agent delegation for codebase exploration are Claude Code features (Domain 3) that solve context management problems (Domain 5). Session management, resume, fork, and clear, are Domain 3 mechanisms that address Domain 5 concerns.

Connection to Domain 4 (Prompt Engineering): The lost-in-the-middle effect is both a prompt engineering concern (where to place instructions) and a context management concern (how position affects reliability). Few-shot examples consume context budget, creating a tension between prompt quality and available context.

Common Exam Patterns in Domain 5

The exam uses several recurring patterns for Domain 5 questions:

The "session gets worse over time" pattern: A session starts accurate and becomes unreliable as it runs longer. The answer almost always involves context management things such as clearing stale content, persisting important facts, or delegating to subagents.

The "missing data looks like absent data" pattern: A downstream agent treats an access failure as a valid empty result. The answer is always structured error responses that distinguish failure from absence.

The "aggregate hides the weak spot" pattern: Overall accuracy looks good, but a specific segment is failing. The answer is stratified sampling by segment.

The "explicit request overridden" pattern: A user asks for a human, and the system tries to resolve the issue first. The answer is always immediate escalation.

The "sentiment as trigger" pattern: The system escalates based on frustration or de-escalates based on a calm tone. The answer is always that sentiment is an unreliable trigger, use concrete conditions instead.

The "conflicting sources" pattern: Two sources disagree, and the report shows one number without attribution. The answer is always to preserve claim-source mappings, annotate the conflict, and use dates to interpret.

Decision Framework for Domain 5 Questions

When you encounter a Domain 5 question, use this framework:

1. Is the problem about context growing too large? → Context management: clear, compact, or delegate.
2. Is the problem about losing specific facts? → Durable facts block, memory tool, or scratchpad.
3. Is the problem about when to involve a human? → Check for concrete triggers: explicit request, policy gap, repeated failure.
4. Is the problem about an error in a pipeline? → Structured errors with category, description, and retryability.
5. Is the problem about accuracy measurement? → Stratified sampling by segment.

6. Is the problem about conflicting information? → Provenance: claim-source mappings with dates.

Key Distinctions the Exam Tests

Concept A	Concept B	The Distinction
Context editing	Compaction	Editing clears specific content; compaction summarizes everything
Durable facts	Passing history	Durable facts are pinned and never summarized; history can be trimmed
Access failure	Valid empty result	Failure means the query did not execute; empty means it executed and found nothing
Immediate escalation	Offered resolution	Explicit human request triggers immediate; frustrated but no request allows offering help first
Sentiment	Concrete trigger	Sentiment is unreliable; concrete conditions (policy gap, repeated failure) are reliable
Aggregate accuracy	Stratified accuracy	Aggregate can hide a failing segment; stratified reveals it
Retention	Retrieval	Retention keeps data in the window; retrieval stores it externally and fetches on demand
Silent suppression	Coverage annotation	Suppression hides the gap; annotation honestly reports it
Invented consensus	Annotated conflict	Consensus fabricates agreement; annotation preserves the disagreement

Worked Example: Full Domain 5 Scenario

Scenario: A customer support agent has been running for 30 minutes, handling a complex billing dispute. The session includes 12 tool calls (account lookup, order history, payment records, refund policy, etc.). The customer mentions a specific charge date (May 2, 2026) early in the conversation. After the agent compacts the conversation to free space, it refers to "the disputed charge" but cannot recall the specific date. The customer then says: "I've explained this three times already, just let me talk to someone."

Domain 5 analysis:

1. **Context management:** The charge date was a durable fact that should have been pinned in a case facts block before compaction. The compaction summary dropped the specific date.
2. **Escalation:** The customer's message "just let me talk to someone" is an explicit request for a human. This triggers immediate escalation, not another attempt to resolve.
3. **Warm handoff:** The escalation should include the case facts (customer ID, disputed charge date, amount), what tools were already used, and what was found.

Correct architecture: Before compaction, save `{customer_id, charge_date: "2026-05-02", amount, policy_result}` to the memory tool. After compaction, re-inject these facts. When the customer requests a human, escalate immediately with the case facts, attempted steps, and a one-line summary.

Wrong patterns: Asking the customer to repeat the date (frustrating). Trying to resolve after the explicit human request (overrides the request). Compacting without saving the case facts (loses the date). Escalating without context (forces the human to start cold).

References for Domain 5: Context Management & Reliability

All links verified against the current Anthropic documentation.

Context Management

- <https://www.anthropic.com/news/context-management>

Context Editing

- <https://docs.claude.com/en/docs/build-with-claude/context-editing>

Memory Tool

- <https://docs.claude.com/en/docs/agents-and-tools/tool-use/memory-tool>

Long-Context Tips

- <https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/long-context-tips>

Reduce Hallucinations

- <https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/reduce-hallucinations>

Tool Use Overview

- <https://docs.claude.com/en/docs/agents-and-tools/tool-use/overview>

Implement Tool Use

- <https://docs.claude.com/en/docs/agents-and-tools/tool-use/implement-tool-use>

Structured Outputs

- <https://docs.claude.com/en/docs/build-with-claude/structured-outputs>

Prompt Caching

- <https://docs.claude.com/en/docs/build-with-claude/prompt-caching>

Be Clear and Direct

- <https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/be-clear-and-direct>

How Claude Code Works

- <https://code.claude.com/docs/en/how-claude-code-works>

Sub-Agents

- <https://code.claude.com/docs/en/sub-agents>

Claude Code Best Practices

- <https://docs.anthropic.com/en/docs/claude-code/best-practices>

CCA-F Official Exam Page

- <https://clau.de/CCAF>